

UNIX/LINUX TEXT PROCESSING

Comprehensive Guide to sed, grep, awk & Regular Expressions

Level:	Intermediate
Focus:	Text Manipulation & Regex
Difficulty:	Advanced (■■■■■)
Updated:	April 27, 2026

Table of Contents

- 1. Conceptos Clave
- 2. Preguntas 7-9: sed Avanzado
- 3. Preguntas 10-12: Manipulacion de Archivos
- 4. Preguntas 13-16: Regex & Busqueda
- 5. Plan de Estudio (7 dias)
- 6. Tips Importantes
- 7. Comandos Git para Guardar en GitHub

1. CONCEPTOS CLAVE

> sed (Stream Editor)

Manipula texto con búsqueda y reemplazo. Sintaxis: **sed 's/BUSCAR/REEMPLAZAR/flags'**

> grep (Global Regex Print)

Busca patrones. Flags: **-i** (case-insensitive), **-w** (palabra exacta), **-c** (cuenta)

> cut

Extrae columnas. Sintaxis: **cut -d'DELIMITADOR' -f COLUMNAS**

> Expresiones Regulares

^ (inicio), **\$** (fin), **[0-9]{5}** (5 dígitos), **\b** (límite palabra)

2. SED AVANZADO (Preguntas 7-9)

P7: Case-Insensitive Replacement

Cambiar 'disabled' a 'enabled' ignorando mayusculas

```
sed -i 's/disabled/enabled/gi' /ruta/archivo.conf
```

Explicacion: El flag **i** hace que sed ignore mayusculas. El flag **g** aplica a toda la linea.

P8: Reemplazo en Rango de Lineas

Cambiar 'enabled' a 'disabled' solo entre linea 500-2000

```
sed -i '500,2000 s/enabled/disabled/g' archivo.conf
```

Clave: El rango **500,2000** limita la operacion a esas lineas especificas.

P9: Caracteres Especiales

Reemplazar `##$2jh//238720//31223` por `$2//23872031223`

```
sed -i 's|##$2jh//238720//31223|$2//23872031223|g' data.txt
```

Tip: Usar **|** como delimitador evita conflictos con **/** en la cadena.

3. MANIPULACION DE ARCHIVOS (Preguntas 10-12)

P10: Mover Linea en Editor vi

Mover linea 1049 a linea 5

```
:1049 ENTER → yy → :5 ENTER → P → :wq ENTER
```

Comandos vi: **yy** (copiar), **dd** (cortar), **P** (pegar arriba), **:wq** (guardar)

P11: Eliminar Primeras 1000 Lineas

Borrar lineas 1-1000

```
sed -i '1,1000d' archivo.txt
```

Alternativa con tail:

```
tail -n +1001 archivo.txt > temp && mv temp archivo.txt
```

P12: Encontrar Linea Unica

file1 y file2 son 99% iguales. Guardar linea diferente en file3

```
diff file1 file2 | grep '^>' | cut -d' ' -f2- > file3
```

O usar comm para encontrar diferencias:

```
comm -3 file1 file2 | grep . > file3
```

4. REGEX & BUSQUEDA AVANZADA (Preguntas 13-16)

P13: Extraer Numero de 5 Digos

Buscar numero con exactamente 5 digitos

```
grep -oE '[0-9]{5}' archivo.txt > numero
```

Flags: **-o** (solo coincidencias), **-E** (extended regex), **{5}** (exactamente 5)

P14: Contar Numeros que Comienzan con 2

Cuantos numeros comienzan con el digito 2?

```
grep -oE '\\b2[0-9]+\\b' archivo.txt | wc -l > count
```

Patron: **\\b** (limite palabra), **2** (primer digito), **[0-9]+** (resto de digitos)

P15: Contar Lineas que Comienzan con 'Section'

Case-insensitive

```
grep -ic '^section' archivo.txt > count_lines
```

Flags: **^** (inicio linea), **-i** (ignore case), **-c** (contar)

P16: Palabra Exacta 'man'

Match: 'keeps man' YES | No match: 'manpath' NO

```
grep -i '\\bman\\b' archivo.txt > man_filtered
```

Clave: **\\b** son limites de palabra. Previenen que 'man' sea parte de otra palabra.

5. PLAN DE ESTUDIO (7 DIAS)

Día	Tema	Objetivos	Ejercicios
1-2	sed 201	Flags: g,i Delimitadores -i (in-place)	P7, P8, P9
3-4	Archivos	vi edits sed delete diff vs comm	P10, P11, P12
5-7	Regex	[0-9]{5} ^ \$ \b grep -E -i -w	P13-16 + Practica

Estrategia Diaria:

1. Repasa los conceptos por 15 minutos
2. Practica CON el documento (abierto)
3. Intenta SIN el documento
4. Si fallas, revisa solo esa seccion

6. TIPS IMPORTANTES

Tip 1: Prueba sin -i primero

```
sed 's/buscar/reemplazar/g' archivo → para ver cambios sed -i 's/buscar/reemplazar/g'
archivo → para aplicar
```

Tip 2: Delimitadores flexibles

```
sed 's|/path/to/dir|/new/path|g' evita conflictos con /
```

Tip 3: Word boundaries

```
grep '\\bman\\b' previene que coincida con 'manpath'
```

Tip 4: Extended regex (-E)

```
grep -E '[0-9]{5}' es mas legible que grep -E '[0-9][0-9][0-9][0-9][0-9]'
```

Tip 5: Caracteres especiales

En sed: # \$ % / → usa | o @ como delimitador

Tip 6: grep -o para coincidir

```
grep -oE 'patron' | wc -l cuenta COINCIDENCIAS (no lineas)
```

7. COMANDOS GIT PARA GUARDAR EN GITHUB

Una vez tengas este PDF, puedes guardarlo en tu repositorio de GitHub con estos comandos:

Pa `git status`

Pa `git add Unix_Linux_Text_Processing_Guide.pdf`

Pa `git commit -m 'Add: Unix/Linux Text Processing Guide'`

Pa `git push origin main`

Ejemplo completo de sesion Git:

```
# Navega a tu repositorio local cd /ruta/a/tu/repositorio # Verifica que cambios tienes
git status # Agrega el PDF git add Unix_Linux_Text_Processing_Guide.pdf # Verifica que
se agrego git status # Crea un commit git commit -m 'Add: Unix/Linux text processing
guide' # Sube a GitHub git push origin main # Verifica resultado git status
```

Notas importantes para Git:

- Asegurate de estar en la rama **main** (o **master** si es antiguo)
- Verifica tu conexion: **git remote -v**
- Primera vez: **git config user.email 'tu@email.com'**
- Mensajes descriptivos son profesionales

RECUERDA

Este NO es un fracaso. Los conceptos de text processing son densos. La mayoría de profesionales de Linux tardo tiempo en dominarlos. Tu ventaja como NOC es que YA SABES troubleshooting. Ahora estan invirtiendo en el siguiente nivel. **Cada linea de comando que comprendas te acerca a ser un Sysadmin Linux profesional.** Guarda este documento. Usalo como referencia. En 2-3 semanas lo recordaras de memoria. **Adelante! Puedes hacerlo!**

Document Created: April 27, 2026 at 02:16
Designed for Linux learners - From NOC to Sysadmin track